

HuggingFace GPT-2とGoogle Colabによるチャットボット開発：理論から実践、そしてキャリアへの応用まで

はじめに：本プロジェクトの意義とレポートの全体像

本レポートは、HuggingFaceのGPT-2モデルとGoogle Colabを活用したチャットボット制作を通じて、単なる技術的な実装にとどまらない、AI開発の全体像と本質的な知見を体系的に解説することを目的としている。この組み合わせは、ローカル環境構築の手間を排し、無料の計算資源を提供することで、誰もが最先端の言語モデルに触れ、その振る舞いを深く理解するための理想的な出発点となる。本報告書は、チャットボットの「なぜ動くのか」という理論的基盤から、Google Colab上での「どのように動かすか」という実践的な実装ガイド、そして「いかにして発展させるか」という応用的な課題解決までを、3つのフェーズに分けて詳細に論じる。読者がこれらのステップを順に追うことで、得られた知識を単発のプロジェクトで終わらせず、自身のスキルとして定着させ、将来的なキャリアへと繋げるための具体的な指針を示す。

第1部：チャットボットの理論的基盤とGPT-2の核心

このセクションでは、チャットボットの応答生成を支える中核技術である大規模言語モデル（LLM）の仕組みを掘り下げ、特にGPT-2のアーキテクチャが持つ特徴を詳述する。

1.1. Transformerモデルの解説：Attentionメカニズムの核心

Transformerモデルは、2017年の論文『Attention Is All You Need』によって導入され、自然言語処理（NLP）分野に革命をもたらした。従来のRNN（再帰型ニューラルネットワーク）が、時系列データをトークンごとに順次処理し、長期的な文脈を捉える際に勾配消失問題を抱えていたのに対し、Transformerは文脈全体を一度に並列処理する「自己アテンション（

Self-Attention) 」機構を採用している¹。この画期的な機構は、文章中の各単語が他のすべての単語とどのような関連性を持つかを動的に計算することで、単語の表面的な意味だけでなく、文脈に応じた適切な解釈を可能にする¹。

自己アテンションの核心は、「クエリ (Query) 」「キー (Key) 」「バリュー (Value) 」という3つのベクトルにある。各トークンはこれらのベクトルを持ち、モデルはクエリとキーのドット積を計算することで、そのトークンが他のどのトークンに「注意」を向けるべきかを決定する¹。この計算結果は、モデルが特定の瞬間にどの情報に焦点を当てるかを重み付けする「アテンション重み」に変換される。最終的に、これらの重みを使ってバリューをスケーリングし、文脈が考慮された新たな表現を生成する。この仕組みにより、Transformerは長文の文脈も効率的に理解し、高品質な文章生成を可能にしている¹。

GPT-2は、このTransformerアーキテクチャの中でも、特に「デコーダのみ」の構造を採用している点に特徴がある²。一般的なTransformerがエンコーダとデコーダの両方で構成され、翻訳などの入力と出力が異なる長さのタスクに適しているのに対し、デコーダのみの構造は、入力シーケンスから次のトークンを予測する、すなわち「テキスト生成」というタスクに最適化されている²。この設計思想が、GPT-2がチャットボットのような対話アプリケーションにおいて、自然で連続的な応答を生成する能力の基盤となっている。

1.2. GPT-2モデル：自己回帰 (Autoregressive) 言語モデルの理解

GPT-2は、自己回帰 (Autoregressive) モデルとして分類される。これは、モデルが自身の出力を次の入力として再帰的に使用し、逐次的にテキストを生成する仕組みを指す⁵。チャットボットの文脈では、ユーザーの入力とこれまでの会話履歴をコンテキストとして与え、次に続くであろう最も可能性の高いトークンを予測する。その予測されたトークンを再び入力シーケンスの末尾に追加し、次のトークンを予測する、というプロセスを繰り返すことで、一連の自然な文章を生成する⁵。このシンプルな「次のトークン予測」という原理が、あたかも人間と対話しているかのような、複雑で流暢な会話体験を生み出す⁵。

初代GPTモデルは、広範なデータで事前学習した後に、特定のタスク (例：翻訳、要約) に対して追加のデータで「ファインチューニング」を行う必要があった³。一方、GPT-2の論文は、ファインチューニングなしの「ゼロショット」設定でも、高いマルチタスク性能を発揮できることを示した⁶。これは、モデルの容量を1.5Bパラメータにまで拡大し、WebTextという膨大かつ多様なデータセットで学習させた結果として達成された⁴。このブレイクスルーは、モデルのサイズと学習データの多様性が、特定のタスクに特化したデータを用意しなくても汎用的な能力を引き出す上で極めて重要であることを示唆した。GPT-2は、単なる高性能な言語モデルではなく、後のLLM開発における「スケールアップのパラダイム」を確立した記念碑的なモデルなのである。

1.3. トークン化の重要性：BPEの役割

自然言語を機械学習モデルで扱うためには、まずテキストをモデルが処理できる数値データに変換する必要がある⁸。この変換を担うのが「トークナイザ」であり、NLPパイプラインにおける最も重要な構成要素の一つである⁸。GPT-2は、このトークン化にByte-Pair Encoding（BPE）というサブワードトークン化手法を採用している⁷。

BPEは、単語を文字のペアに分解し、出現頻度の高いペアを一つの新しい「サブワード」トークンとして結合していくことで語彙を構築する⁹。この手法の利点は、未知の単語（Out-of-Vocabulary、OOV）問題に柔軟に対応できる点にある。例えば、「unbelievable」という単語を「un」と「believable」というサブワードに分割すれば、たとえこの単語が学習データになくても、構成要素から意味を推論できる可能性が高まる⁹。GPT-2は、このBPEによって50,257という語彙数を効率的に管理し、膨大なテキストデータから学習した知識を、より汎用的な単位で表現している⁹。

以下の表は、主要なトークン化手法の特徴を比較したものである。

手法	単位	特徴	メリット	デメリット
単語	単語	テキストを単語単位で分割	自然言語の単位に近く、直感的	語彙数が爆発的に増加、未知語（OOV）問題が発生
文字	文字	テキストを文字単位で分割	語彙数が非常に少ない、OOV問題なし	長い単語や文脈の処理が非効率
サブワード（BPE）	単語の一部	単語と文字の中間の単位。頻出ペアを結合	未知語に柔軟、語彙数を抑制	語彙の選択がモデルの性能に影響

表1：トークン化手法の比較

BPEの採用は、GPT-2が未知の単語や複雑な構文にも対応できる、汎用的かつ堅牢なモデルであることを裏付けている⁷。

第2部：実践環境の構築：Google ColabとHuggingFaceの活用

理論的な理解を深めた上で、実際にチャットボットを動かすための実践的な環境構築について解説する。

2.1. Google Colabの環境セットアップ：無料でGPUを活用する

Google Colabは、ブラウザ上で直接Pythonコードを実行できる対話型プログラミング環境であり、ローカルマシンに開発環境を構築する手間を省くことができる¹⁰。これは、特に機械学習プロジェクトの初学者にとって、環境構築に時間を費やすことなく、すぐにコードの学習に集中できるという大きな利点をもたらす¹⁰。

GPT-2のような大規模モデルの推論や学習には、GPU（Graphics Processing Unit）が不可欠である。Google Colabは、このGPUを無料で利用するサービスを提供している¹⁰。GPUを有効化するには、ノートブック上部のメニューから「ランタイム」を選択し、「ランタイムのタイプを変更」をクリックする。ポップアップ画面で「ハードウェア アクセラレータ」を「GPU」に設定することで、GPUを割り当てて利用することが可能となる¹¹。

ただし、Google Colabの無料版には、セッションが12時間で切断されるという制限がある¹¹。この制限は一見すると制約に思えるが、開発プロセスにおいてはむしろ良い習慣を促す。すなわち、セッションが切断されると、それまでアップロードしていたファイルやインストールしたライブラリはすべてリセットされるため、定期的なコードの保存と、効率的で計画的な開発を習慣づけるきっかけとなる¹¹。この制約をポジティブに捉え、作業の区切りごとにコードをGitなどのバージョン管理システムにコミットする習慣を身につけることは、プロフェッショナルな開発者になるための重要な第一歩となる。

2.2. HuggingFaceライブラリの基本：モデルとトークナイザーの扱い方

HuggingFaceは、最先端の言語モデルやツールをオープンソースで提供するプラットフォームであり、その中心となるのがtransformersライブラリである¹⁴。このライブラリは、モデルと

トークナイザを簡単にロード・利用するためのAPIを提供しており、AI開発のワークフローを劇的に効率化する¹⁴。

Google Colab上でHuggingFaceライブラリを使用するためには、まず必要なパッケージをインストールする必要がある。これは、コードセルに!pip install transformersのようなコマンドを実行することで行える¹⁶。もし特定のモデルに依存する追加のライブラリがある場合は、そのモデルのModelCardに記載されているインストール手順に従う¹⁶。

HuggingFace Hubは、膨大な数の事前学習済みモデルが公開されている「モデルの宝庫」である¹⁴。チャットボットを制作する際には、左上の検索バーで「Japanese」や「text generation」といったキーワードで絞り込み、目的に合ったモデルを見つけることができる¹⁶。モデルを使用する前に、ModelCardでライセンスや使用許諾を確認し、必要に応じて同意することも重要なプロセスである¹⁶。HuggingFaceのエコシステムは、単に便利なライブラリやモデルの集合体ではなく、AI開発のワークフロー全体を標準化し、開発者間のコラボレーションを促進するプラットフォームとして機能している。このエコシステムを理解することで、ユーザーはGPT-2に留まらず、今後登場するあらゆるモデルやタスクに応用可能な普遍的な知見を獲得できる。

2.3. HuggingFace pipeline APIによるチャットボットのプロトタイプ制作

HuggingFaceのpipeline APIは、チャットボット制作における複雑なプロセスを抽象化してくれる高水準なインターフェースである¹⁸。これを利用することで、モデルのロード、トークナイザによる入力テキストの前処理、推論、そして出力の後処理といった一連の処理を、わずか数行のコードで実現できる¹⁸。

チャットボットを実装する際は、TextGenerationPipelineを「チャットモード」で利用することが推奨されている¹⁹。これは、ユーザーとアシスタントのメッセージ履歴を辞書のリスト形式で管理する、現代のチャットボット開発における基本的なプログラミングパターンである¹⁹。

Python

```
import torch
from transformers import pipeline

# パイプラインを初期化
# GPT-2モデルの推論にはGPUが必要なため、device_map="auto"を指定
pipe = pipeline(task="text-generation", model="gpt2", device_map="auto")
```

```
# 最初の対話履歴を定義
chat_history = [
    {"role": "user", "content": "こんにちは、自己紹介してください。"}
]

# モデルにテキスト生成を依頼
response = pipe(chat_history)

# 生成された応答を表示
print(response["generated_text"][-1]["content"])
```

このコード例のように、ユーザーのメッセージを辞書のリストとしてpipelineに渡すことで、モデルはこれまでの会話の文脈を考慮した応答を生成する¹⁹。GPT-2は「事前学習済み（pretrained）」モデルであり、

TextGenerationPipelineが特に「指示応答（instruct-tuned）」モデルに最適化されている点には注意が必要である¹⁹。この違いを理解することは、ユーザーがGPT-2のファインチューニングを深く学ぶ道と、より手軽な現代のモデルでプロトタイプを迅速に構築する道の、どちらに進むべきかを判断する上で重要な出発点となる。

第3部：チャットボットの高度化と課題解決

基本的なプロトタイプから一歩進み、より自然で実用的なチャットボットを構築するために直面する課題と、その解決策について解説する。

3.1. 対話の「記憶」：コンテキストウィンドウ管理の挑戦

LLMは、人間のように無限の記憶を持つわけではない。モデルが一度に処理できる入力テキストの長さには厳格な上限があり、これを「コンテキストウィンドウ」と呼ぶ²⁰。このコンテキストウィンドウには、システムプロンプト、過去のユーザーとアシスタントの全メッセージ、そして現在のユーザー入力とモデルの応答もすべて含まれる²⁰。コンテキストウィンドウの制限は、チャットボットの「短期記憶」を物理的・概念的に定義するものであり、この上限を超えるとモデルは過去の文脈を「忘れて」しまい、応答の質が著しく低下する²⁰。この技術的な制約を理解することは、チャットボット開発における「記憶管理」の重要性を根本的に理解する上で不可欠である。

3.2. 効率的な対話履歴の管理戦略

コンテキストウィンドウの制限を克服し、長時間にわたる自然な会話を維持するためには、対話履歴を効率的に管理する戦略が必要となる。最もシンプルな手法は、「履歴のトリミング」である。これは、最も古い会話から順にメッセージを削除し、常に直近のNターン分のみをコンテキストとして保持する手法である²⁰。

より高度な手法として、「対話の要約」がある。これは、古い会話の履歴をモデル自身に要約させ、その要約文と直近の会話履歴を組み合わせることで、トークン消費を抑えつつ重要な文脈を維持する手法である²¹。このアプローチは、モデルの生成能力を「自己管理」というメタレベルのタスクに応用する優れた例であり、ユーザーに深い示唆を与える。

以下の表は、主要な対話履歴管理戦略の特徴と、そのメリット・デメリットを比較したものである。

戦略	特徴	メリット	デメリット
単純な連結	全ての会話履歴を時系列順に連結	実装が最も容易、完全な履歴を保持	トークン消費が急増、すぐにコンテキスト制限に達する
履歴のトリミング	直近のNターンのみを保持	シンプルで効率的、トークン消費を抑制	重要な過去の文脈が失われる可能性がある
対話の要約	古い会話を要約し、文脈を保持	長い会話でも文脈を維持、トークン消費を大幅削減	要約の質が応答に影響、実装が複雑
外部データベース（RAG）	関連情報を外部データベースから取得	記憶容量を実質的に無限に拡張、正確な情報を提供	実装が最も複雑、検索の精度が性能を左右

表2：対話履歴管理戦略の比較

3.3. 応答品質の向上：GPT-2のファインチューニングと次世代モデル

GPT-2を特定の目的（例：特定のキャラクターを演じる、専門的な知識について対話する）に特化させるには、ファインチューニングが有効な手法となる⁵。ファインチューニングとは、広範なデータで学習済みのモデル（GPT-2）を、特定のタスクに特化した少量のデータ（例：対話データセット）で再学習させるプロセスである³。このプロセスでは、ユーザーの入力とモデルの応答を、

[end_of_turn]のような特殊なトークンで区切ることで、モデルが対話の構造を学習できるようになる⁵。

これは、GPT-2が広範なデータでの「転移学習」によって獲得した膨大な知識を、ファインチューニングというプロセスを通じて、特定の対話タスクに効率的に応用する作業である³。この学習パラダイムは、AI開発における効率的なモデル構築を象徴しており、今後の学習においても応用すべき重要な概念である。

また、HuggingFace Hubには、microsoft/DialoGPTのように、対話に特化するためにファインチューニングされたGPT-2の派生モデルも存在している⁵。こうしたファインチューニング済みモデルの存在は、ゼロからモデルを学習させる必要性を減らし、開発効率を飛躍的に向上させる。用途に応じたモデルを適切に選択することは、プロジェクト成功の鍵となる。

第4部：効率的な学習とプロジェクトのポートフォリオ化

チャットボット制作は、単なる技術的な演習にとどまらない。その成果を公開し、自身のスキルを証明することで、学習をキャリアへと繋げることができる。

4.1. プロジェクトの成果を公開する：GitとGitHubの活用

Gitは、ファイルの変更履歴を管理するための分散型バージョン管理システムである²⁴。Gitを使うことで、開発者はいつ、誰が、何を修正したかを一元的に追跡できる²⁴。これにより、たとえ誤ってコードを上書きしてしまっても、簡単な操作で過去の任意のバージョンに簡単に戻すことが可能になる²⁴。

GitHubは、Gitリポジトリをホスティングするウェブサービスであり、コードを世界中の開発者

と共有するためのプラットフォームである²⁶。プロジェクトのコードをGitHubリポジトリに公開することで、それは単なるコードの保存場所ではなく、自身の技術力やプロジェクト管理能力、そして学習のプロセスそのものを可視化する「デジタルポートフォリオ」となる²⁷。

GitHub上でリポジトリを作成し、基本的なGitコマンド（git add、git commit、git push）を使ってコードを公開することは、今日のソフトウェア開発者にとって必須のスキルである²⁴。このプロセスを通じて、開発者は単にチャットボットを動かすだけでなく、プロフェッショナルな開発ワークフローを体験することができる。

4.2. 読み手の心を掴む：README.md作成のベストプラクティス

GitHub上のプロジェクトの成功は、そのコードの品質だけでなく、読み手の心を掴む「README.md」の品質に大きく左右される¹⁹。README.mdはプロジェクトの「フロントドア」であり、訪問者がプロジェクトの目的、使い方、そして価値を瞬時に理解するための唯一の場所である²⁸。優れたREADMEは、訪問者を単なる閲覧者から、プロジェクトのユーザー、さらには共同開発者に変える力を持っている²⁸。

以下の表は、チャットボットプロジェクトの魅力を最大限に引き出すために、README.mdに含めるべき必須要素のチェックリストである。

セクション	必須要素	概要
プロジェクト概要	タイトルと説明	簡潔で魅力的なプロジェクト名と、その目的を1～2文で説明する ²⁸
プロジェクト詳細	概要と特徴	プロジェクトのコア機能と主要な特徴をリスト形式で説明する ²⁸
技術スタック	言語、ライブラリ、API	使用した主要なプログラミング言語、フレームワーク、ライブラリ、外部APIを明記する ²⁸
導入手順	前提条件とインストール	プロジェクトをローカルで動かすための前提条件と、ステップごとのインストー

		ル手順を詳述する ²⁸
使用方法	コマンドと例	実際の使用例をコードスニペットやスクリーンショットで示す ³⁰
ライセンスと貢献	ライセンス、貢献方法	プロジェクトのライセンスを明記し、他の開発者が貢献する方法を説明する ²⁹

表3 : プロジェクトREADME.md必須要素チェックリスト

このチェックリストに基づき、プロジェクトのビジョンと価値を戦略的に伝えることで、GitHubを単なる技術ドキュメントの置き場ではなく、自身の能力をアピールするための強力なプロモーションツールとして活用できる。

結論 : 学習ロードマップと次のステップ

本レポートは、HuggingFaceのGPT-2とGoogle Colabを組み合わせたチャットボット制作が、AI開発の深い学習体験を提供することを示した。Transformerモデルの理論的背景から、pipeline APIを用いた実践的な実装、そしてコンテキスト管理といった高度な課題までを網羅的に解説した。

このプロジェクトを通じて得られた知識と経験は、今後の学習ロードマップの基盤となる。次のステップとしては、より高度なチャットボット開発に挑戦することが考えられる。例えば、モデルのAPIを利用し、外部知識ベースと連携させるRAG (Retrieval-Augmented Generation) のような技術を実装することで、モデルの応答が持つ「記憶」の限界を根本的に拡張することができる²²。また、GPT-2とは異なるアーキテクチャを持つモデル（例：BERT）を探索することで、言語モデルの多様なアプローチを学ぶことも有効である。

今回のプロジェクトを通じて培われた、理論的な理解、実践的な実装能力、そしてプロジェクトの公開とドキュメンテーション能力は、AIエンジニア、データサイエンティスト、あるいは機械学習エンジニアとしてのキャリアパスを切り開く上で不可欠な資産となる。本レポートが、読者のAI開発における持続的な学習と成長の出発点となることを切に願う。

引用文献

1. Transformerモデルとは - IBM, 9月 9, 2025にアクセス、
<https://www.ibm.com/jp-ja/think/topics/transformer-model>
2. Transformer（トランスフォーマー）とは？ 深層学習の仕組みや特徴・応用モデルをわかりやすく解説, 9月 9, 2025にアクセス、
https://aismiley.co.jp/ai_news/what-is-transformer-deep-learning/
3. GPT-2 - Wikipedia, 9月 9, 2025にアクセス、 <https://ja.wikipedia.org/wiki/GPT-2>
4. GPT-2を理解する | 論文メモ: Language Models are Unsupervised Multitask Learners, 9月 9, 2025にアクセス、
<https://www.bioerrorlog.work/entry/gpt-2-paper>
5. gpt2-chatbot - Kaggle, 9月 9, 2025にアクセス、
<https://www.kaggle.com/code/pinooxd/gpt2-chatbot>
6. GPT-2を理解したい - Zenn, 9月 9, 2025にアクセス、
<https://zenn.dev/sunbluesome/articles/775ffd67fb7454>
7. 【GPTシリーズを理解する】 GPT-2の論文を詳細解説！ - DeepSquare, 9月 9, 2025にアクセス、 <https://deepsquare.jp/2023/07/gpt-2/>
8. huggingface.co, 9月 9, 2025にアクセス、
<https://huggingface.co/learn/llm-course/ja/chapter2/4#:~:text=%E3%83%88%E3%83%BC%E3%82%AF%E3%83%8A%E3%82%A4%E3%82%B6%E3%81%AFNLP%E3%83%91%E3%82%A4%E3%83%97%E3%83%A9%E3%82%A4%E3%83%B3,%E5%A4%89%E6%8F%9B%E3%81%99%E3%82%8B%E5%BF%85%E8%A6%81%E3%81%8C%E3%81%82%E3%82%8A%E3%81%BE%E3%81%99%E3%80%82>
9. Huggingface Transformers 入門 (8) - トークナイザー | npaka - note, 9月 9, 2025にアクセス、 <https://note.com/npaka/n/nb08941a36c8b>
10. Google Colaboratory（Colab）とは・使い方などの基礎知識を初心者向けに解説 - データミックス, 9月 9, 2025にアクセス、
<https://datamix.co.jp/media/datascience/what-is-google-colaboratory/>
11. Google colaboratoryの使い方を5つの基本操作でわかりやすく解説 - Alsmiley, 9月 9, 2025にアクセス、
https://aismiley.co.jp/ai_news/how-to-use-google-colaboratory/
12. Google Colabの知っておくべき使い方 - Google Colaboratoryのメリット・デメリットや基本操作のまとめ, 9月 9, 2025にアクセス、
<https://www.codexa.net/how-to-use-google-colaboratory/>
13. Google Colabを使って最新AI論文のコードを実行するためのConda仮想環境構築ガイド - note, 9月 9, 2025にアクセス、
https://note.com/holyday_mylife/n/n3738ac5f6394
14. 什么是Hugging Face Transformers？ - Azure Databricks | Microsoft Learn, 9月 9, 2025にアクセス、
<https://learn.microsoft.com/zh-cn/azure/databricks/machine-learning/train-model/huggingface/>
15. 什么是Hugging Face Transformers？ - Azure 文档, 9月 9, 2025にアクセス、
<https://docs.azure.cn/zh-cn/databricks/machine-learning/train-model/huggingface/>
16. HuggingfaceとColabで始める簡単機械学習 #自然言語処理 - Qiita, 9月 9, 2025にアクセス、 <https://qiita.com/menon/items/51f586a04c6595f1a6c3>
17. Google Colabで手を動かして覚える、Pythonライブラリの使い方 - 2024/12/14

- [Schoo], 9月 9, 2025にアクセス、 <https://schoo.jp/matome/article/1817>
18. Pipelines - Hugging Face, 9月 9, 2025にアクセス、
https://huggingface.co/docs/transformers/main_classes/pipelines
 19. Chat basics - Hugging Face, 9月 9, 2025にアクセス、
<https://huggingface.co/docs/transformers/conversations>
 20. AI Chatbots: Memory (Part 2) | The Context Window — Your Bot's Short-Term Memory, 9月 9, 2025にアクセス、
<https://medium.com/@HuggyMonkey/chatbots-memory-part-2-the-context-window-your-bots-short-term-memory-687afcc4d83a>
 21. AIとの会話を長く楽しむために（補足） | ゆうさく - note, 9月 9, 2025にアクセス、
https://note.com/fresh_sedum5880/n/n3ef2f319d883
 22. Mastering LLM Memory: A Comprehensive Guide - Strongly.AI, 9月 9, 2025にアクセス、
<https://www.strongly.ai/blog/mastering-llm-memory-a-comprehensive-guide.html>
 23. js-lee-AI/emotional-chatbot_gpt2: interactive and emotional ... - GitHub, 9月 9, 2025にアクセス、
https://github.com/js-lee-AI/emotional-chatbot_gpt2
 24. 【入門】Gitとは？できることや使い方、GitHubとの違いをわかりやすく解説 - カゴヤ・ジャパン, 9月 9, 2025にアクセス、
<https://www.kagoya.jp/howto/it-glossary/develop/git/>
 25. Git入門：初心者向けの基本操作と概念 - Zenn, 9月 9, 2025にアクセス、
https://zenn.dev/kthrlab_blog/articles/9a304808e4ad8b
 26. GitHubのウェブサイト/ポートフォリオの作り方 - Reddit, 9月 9, 2025にアクセス、
https://www.reddit.com/r/github/comments/1gxr8bk/how_to_create_github_website_portfolio/?tl=ja
 27. readme-template · GitHub Topics, 9月 9, 2025にアクセス、
<https://github.com/topics/readme-template>
 28. GitHub Readme Template: For Personal Projects - DEV Community, 9月 9, 2025にアクセス、
<https://dev.to/sumonta056/github-readme-template-for-personal-projects-3lka>
 29. How to write a great README - Hacker News, 9月 9, 2025にアクセス、
<https://news.ycombinator.com/item?id=36773022>
 30. The Ultimate Guide to Writing a Great README.md for Your Project | by kc-clintone, 9月 9, 2025にアクセス、
https://medium.com/@kc_clintone/the-ultimate-guide-to-writing-a-great-readme-md-for-your-project-3d49c2023357